Project Iteration 1.1: Initial Vision and Planning

Course: DIGT 2107, Practice of Software Development, Fall Term 2026
Instructor: Dr. May Haidar
Due: September 20, 2026

---

1. Introduction

Project Name: SplitEven
Team Number: Team 2
Team Members:

Full name	Student ID	YorkU email
Adam Bridges		
Edward Drobnis		
Francis Kafieh		
Claire Bhagat		

---

2. Project Vision

Vision Statement

Our project seeks to develop a desktop expense sharing application that addresses the difficulty of tracking who paid for what, and who owes whom, in a group that shares costs, which is often faced by roommates, trip groups, and small clubs. By providing expense logging, flexible splitting, automatic balance tracking, a settle up feature, and spending summaries, the system will help users keep an accurate record of shared spending and pay each other back without arguments. The software will be built using Agile principles, iteratively refining the solution based on feedback.

Problem Statement

Groups that share costs usually track them in a group chat or a spreadsheet. Roommates split rent and utilities, friends share costs on a trip, and clubs pay for supplies. Both methods cause the same problems:

- Uneven splits are especially messy when an expense needs to be divided by percentage or only among certain members, often requiring people to round amounts manually or disagree about the correct split.
- Balances also have to be worked out by hand, meaning someone has to add everything up for each member and recalculate it whenever a new expense is added.
- Once people start paying each other back, payments can also get lost in the conversation or spreadsheet, leaving the record out of sync with what is actually owed.
- Even finding basic information can be difficult, such as figuring out how much the group spent on groceries last month without scrolling through every expense.

This causes arguments between people who are otherwise happy to share costs. The group does not need a banking app. It needs a simple record that splits costs the same way every time and always shows who owes whom.

Target Users

The primary users of the application are people who regularly share expenses with a small group and need a simple way to track and settle those costs. This includes roommates managing recurring household expenses such as rent, utilities, and groceries; friends travelling together who share costs for accommodation, food, transportation, and activities; and members of small clubs or student organizations who purchase supplies or other shared expenses. Within these groups, some users may take on more of an organizer role by setting up the group and managing its members, while all members can record expenses, check their balances, and record payments. The application is designed for users who want to quickly understand what they owe or are owed without manually calculating balances.

User	**Role and context**
Group organizer	Sets up the group, adds members, and decides how costs are normally split. Usually the person who already keeps the spreadsheet, such as a lead roommate, a trip planner, or a club treasurer.

User	Role and context
Group member	Adds the expenses they paid for, checks their own balance, and records money they have paid back. Wants a quick answer to “what do I owe?” without doing any math.

Project Goals

1. Build a desktop application with a clear JavaFX interface that someone without technical skills can use right away.
2. Support equal splits, custom amounts, and percentage splits, and keep the balances correct in all three cases.
3. Update each member's balance automatically whenever an expense or a payment is added.
4. Provide a settle up screen that records payments between members and keeps the remaining balances correct.
5. Let the group look back at its spending, filtered by member, category, and date range.
6. Keep all data on the user's own computer, with no accounts, no server, and no bank connections.
7. Build the app in stages, showing each stage in the repo and adding any feedback to the backlog.

---

3. High Level Features

1. Group and Expense Management (GE)

Description: Creating a group, adding and removing members, and adding expenses with an amount, a payer, a date, a category, a note, and a split method. This also covers editing and deleting entries, and the checks that keep the records correct: the amount has to be positive, a payer has to be chosen, and custom splits have to add up to the full amount.

How it fits the vision: Everything else in the app depends on expenses being recorded first.

How it fits the users: Organizers set the group up once and keep the member list current. Members add an expense in a few seconds while the details are fresh, so the record no longer depends on someone remembering later.

2. Balances and Settling Up (BS)

Description: Working out what each member owes or is owed based on the full expense list, showing who owes whom, and recording payments between members so the balances update. This part of the app applies the group's splitting rules, handles uneven splits and payments, and always shows the current situation.

How it fits the vision: This is one of the main functionalities of the app. It replaces the math that people currently do by hand, which is what causes most of the arguments.

How it fits the users: Members get a straight answer to "what do I owe?" without doing any math. Organizers stop being the group's accountant, because the app keeps the running total and shows who should pay whom.

3. User Experience and Interface (UI)

Description: A main screen showing the group's current balances, a table of past expenses, forms for adding an expense or recording a payment, and clear feedback after every action. This includes error messages shown next to the field that caused the problem, confirmation messages when something works, and screens that can be used with the keyboard.

How it fits the vision: If adding an expense takes longer than sending a message about it, the group will go back to the group chat so this is meant to simplify the process.

How it fits the users: Members use the app in short visits, usually right after paying for something. Simple forms and clear error messages let them finish in under a minute. An organizer can hand the app to a new member without having to explain it.

4. Reporting and Summaries (RS)

Description: Filtering past expenses by member, category, and date range, showing totals per category and per member for a chosen period, and saving a plain text summary of the group's current balances that can be shared with the others.

How it fits the vision: Groups need to review their shared spending as well as record it, both to settle up and to see whether their costs are reasonable.

How it fits the users: Organizers need totals when the group settles up or when someone questions a cost. Members want to check one category or one month without reading through every entry.

---

4. Iteration Plan

Iteration 1 (Weeks 1 to 2, September 9 to 20): Initial Vision and Planning

Deliverables

- Project vision document (this document)
- Initial feature categories and high level functionality overview
- Basic repository setup, including the branch structure and a README

Iteration 2 (Weeks 3 to 5, September 21 to October 11): Requirements and User Stories

Deliverables

- Detailed user stories with acceptance criteria, priorities, and story points
- Functional and non functional requirements document
- Use case diagrams and descriptions
- Updated feature backlog, in priority order
- Initial prototype covering the basic structure and one main feature
- Updated plan and change log

Iteration 3 (Weeks 6 to 9, October 19 to November 8): Initial Development and Testing

Deliverables

- Unit tests for the splitting and balance code
- Initial prototype covering the high priority user stories
- Testing plan and report, including a table linking each requirement to its test cases

---

5. Initial Setup and Repository Structure

GitHub Repository Link: https://github.com/Edddro/DIGT2107-SplitEven

Repository Structure

- **main** branch: stable releases. Each deliverable is tagged here as **ITR1.1**, **ITR1.2**, and so on.
    - **dev** branch: active development, where finished work is combined.
        - **feature** branches: one per user story
            - **feat/ab/UI-###-short-description**, and merged into **dev** through a PR
            - Example: {UI = User Experience and Interface} - {### = (User Story) 001} - {balance-dashboard}

Directories

```
README.md
src/
├── main/java/      # Application code
└── test/java/      # JUnit tests
docs/
├── deliverables/   # Submitted PDFs
├── requirements/   # Requirements and test tracking
├── testing/        # Test documentation
└── design/         # Diagrams and illustrations
sprint-log.md       # Iteration goals, results, retrospectives
```

Planning tools

We will use GitHub Issues and a Projects board. We will take advantage of the milestone feature and use one per iteration.

Development environment

The application will be developed using Java and JavaFX, with JUnit 5 used for unit testing. The project will use Gradle for build and dependency management once it is covered in the course. For local persistence, the initial plan is to store application data in CSV files, keeping the application self-contained without requiring a server, user accounts, or an internet connection. The storage approach will be reviewed during Iteration 1.2 as the data model and requirements are finalized to ensure it can reliably store groups, members, expenses, and payments.

---

7. Next Steps

For Iteration 1.2, the team will:

1. Develop detailed user stories with acceptance criteria, priorities, and story points, and break each story into smaller development tasks with time estimates.
2. Define functional and non-functional requirements and create a table linking each requirement to its corresponding user story.
3. Create the use case diagram and descriptions for the application's main interactions, including adding an expense, viewing balances, and settling up.
4. Develop the initial prototype, starting with the smallest working version that can create and save an expense and calculate and display the resulting balance.
5. Review feedback from Iteration 1 and update the backlog and project plan to reflect any changes to the requirements, priorities, or implementation approach.
6. Review the material posted under Module 2 to ensure the team/project are aligned with its requirements.